ChatGPT

Automated Data Harvester Design for the Concept2Cure Platform

Overview and Goals

The **Concept2Cure** platform requires a robust, **automated data harvester** that continuously ingests files from various sources, processes them into structured data, and updates an internal knowledge base (the "data intelligence cortex"). The goal is to design a **multi-service pipeline** (a farm of "data farmer" microservices) that performs the following in an ongoing loop: fetch new files, **park** (store) them temporarily, **scrub** and extract atomic data from those files, discard the bulky raw artifacts, save the cleaned data into the platform's data model, and finally populate the **intelligence cortex** with this information. This design must emphasize continuous operation, scalability, and reliability so that data flows **constantly and reliably** into the system for analysis [1].

Key objectives for the harvester design include:

- **Continuous Ingestion:** The system should run continuously (or on a schedule) to capture new or updated files from all relevant sources as soon as they are available.
- **Decoupled Processing:** Ingestion, processing, and storage stages are separated into independent services ("farmers") to improve scalability and fault tolerance.
- **Data Cleaning & Transformation:** Extracted data must be cleaned and normalized ("scrubbed") into a standardized format that fits the Concept2Cure data model.
- **Efficient Storage:** After extraction, large raw files can be archived or discarded to conserve storage, while the essential structured data is retained in the database.
- **Intelligence Cortex Update:** The refined data populates the platform's "cortex," a centralized intelligence layer (e.g. a knowledge graph or search index) that fuses all incoming information for advanced querying and AI analysis.

By achieving these goals, the harvester will ensure that *every pulse of the organization's data ecosystem brings critical information to where it's needed*, kept up-to-date for analysis and decision-making [1].

High-Level Architecture Overview

The automated harvester is designed as a **pipeline of microservices** (farmers) orchestrated to perform Extract-Transform-Load (ETL) in stages. Each stage is handled by a dedicated service or module, and they communicate through well-defined APIs or asynchronous messaging. This microservices approach ensures the system is modular, scalable, and resilient – each "farmer" service can run independently and be scaled or updated without affecting others [2].

Figure: A typical data ingestion pipeline architecture, from source data to processing and storage. In our design, multiple microservice "farmers" handle different stages (source extraction, transformation, loading), enabling continuous and scalable data flow. [3] [4]

At a high level, the **data harvester pipeline** operates as follows (each step corresponds to one or more microservice components):

1. **Source Identification & Retrieval:** The harvester begins by identifying and monitoring all relevant data sources. These can include structured sources (databases, APIs), semi-structured sources (JSON/XML feeds), or unstructured files (documents, PDFs, images) [5]. For each source, a **Harvester Service** (or "farmer") is responsible for connecting to that source (e.g. via API calls, web scraping, database queries, or file system watchers) and **pulling new files** or data at regular intervals. This could be implemented with scheduled jobs or listeners – for example, polling an API or an FTP server for new uploads. The key is to **constantly fetch incoming data** as it appears, ensuring no update is missed. Each harvester service immediately **parks** (stores) the raw files or data it retrieves into a **central staging area** (such as a cloud storage bucket or a message queue) for further processing. This staging decouples data retrieval from processing speed, so farmers can continue fetching new files without waiting on downstream processing. *(In practice, tools like Apache NiFi can automate data flows from diverse sources in this manner [6], or custom microservices can be written for each source.)*

2. **Data Extraction & Scrubbing:** Once a raw file is parked in staging, a **Processing Service** picks it up for extraction and cleaning. This service parses the artifact to extract all **data atoms** – the granular pieces of information our platform cares about. For example, if the file is a research PDF or a regulatory document, the processing service might use text parsers or NLP to extract key fields, figures, and insights. If it's structured (like JSON/CSV), it maps the fields to our unified schema. During this extraction, the data is **transformed and cleaned**: the service will normalize formats (dates, units, terminology), remove duplicates or irrelevant sections, and ensure consistency with the target data model [7]. This step is essentially the "Transform" in ETL – converting raw input into standardized, quality data. By the end of this stage, the file's useful content has been distilled into a structured form (e.g. Python dictionaries, JSON records, or database-ready records) representing the **atomic data**. Any necessary validation checks occur here as well – e.g. ensuring mandatory fields are present and values fall in expected ranges – to maintain high data quality [4].

3. **Raw File Disposal:** After successful extraction and scrubbing, the large raw artifact (PDF, image, etc.) is no longer needed for immediate use. The system can then **dispose of or archive the raw file**. In practice, this might mean moving the file to long-term cold storage or deleting it if retention isn't required. This keeps the active storage footprint small and the pipeline efficient. (For traceability, a file's metadata like source and retrieval time can be retained in a log.) Disposing of the raw artifact at this point ensures that the platform isn't cluttered with heavy files after their data has been mined. It also forces all consumers to rely on the cleaned data in the database (single source of truth) rather than potentially re-parsing raw files.

4. **Data Loading into Storage:** The extracted, clean data is now fed into the **Concept2Cure data model storage**. This typically involves a **Load Service** or database connector API that takes the structured data and inserts it into the platform's database or data lake. The target storage could be a relational database (for structured transactional data), a document database (for semi-structured data), or a data lake/warehouse for large-scale analytics. In many cases, a combination is used: for example, structured core data might go into a SQL database, while large text or JSON might be stored in a NoSQL or search index. The loading step writes the data into the appropriate tables/ collections according to the schema. Before finalizing, the system can perform a **data validation**

step – ensuring that all required fields are loaded and that referential integrity or schema constraints are satisfied [4]. Upon successful load, the data is now **persisted in the platform's data store**, ready to be used by other services or users. This completes the main ETL cycle (Extract, Transform, Load). The database now contains the up-to-date facts extracted from the latest files [8].

5. **Intelligence Cortex Update:** The final step is to update the **Concept2Cure Intelligence Cortex** – the central knowledge repository that powers the platform's analytics and AI. Once new data is in the database, another service (or a function of the load service) **indexes and integrates this data into the cortex**. The cortex could be realized in different ways, depending on the platform's design: for example, it might be a **knowledge graph** that links new data points to existing biomedical ontologies, or a **semantic search index** that allows natural language queries over all ingested documents. In some architectures, this involves transforming the data into a graph format or updating a specialized index. *(For instance, one approach is to convert structured data to an RDF triple store, enabling semantic queries and reasoning – an RDF knowledge base can store enriched data for advanced AI analysis* [9] *.)* In other cases, the cortex might involve updating machine learning feature stores or vector databases (for embedding-based search on unstructured text). The **Cortex Update Service** ensures that any new information from the harvested file is **fused into the collective intelligence** of the platform. After this step, the data is not only stored but also **connected and accessible** for insights – users or AI agents can query the cortex to find relationships, answers, and generate reports.

Each of these steps is handled by independent but connected "farmer" services in our design. By breaking the pipeline into microservices, we gain several benefits: **asynchronous buffering** between steps, fault isolation, and the ability to **scale components independently** [10]. For example, if extraction is CPU-intensive, we can scale out more extractor instances without modifying the other parts. If a source's fetch rate increases, we can run more harvester instances for that source. This decoupling is often achieved by using a **message queue or event bus** between stages: one service drops a message or file reference when it's done (e.g., "file X is ready for processing"), and the next service picks it up when ready. Using message-queue-based decoupling ensures producers and consumers (our microservices) are loosely coupled and can run at their own pace, buffering any spikes in workload [11] [12]. Technologies like **Kafka** or **RabbitMQ** (or cloud services like AWS SQS) could serve this role, enabling an **asynchronous, scalable pipeline** where no single component bottlenecks the flow.

Core Components ("Farmers") and Their Roles

In summary, the automated harvester consists of several core **microservice components**, each with a specific role in the pipeline. These can be thought of as a fleet of data farmers working together:

- **Source Harvester Services:** These are small, dedicated services for each data source (or each type of source). A harvester service knows how to connect to a source and retrieve files or data. For example, one harvester might call an external API to fetch new clinical trial documents, while another watches a file directory for lab report uploads. They run on schedules or event triggers, and expose an API (or use internal triggers) to report new data. When a new artifact is fetched, the harvester either pushes it to a central storage (like an object store or database) and/or sends a message (e.g., posts a JSON metadata to a queue) to signal that processing can begin. These services are stateless (or maintain minimal state like last checkpoint) so they can run continuously and in

parallel. If a harvester fails or the source is temporarily down, it doesn't stall the whole pipeline – other harvesters continue farming other sources.

- **Processing & Scrubbing Service:** This service (or set of services) handles the heavy lifting of parsing files and extracting data. It may be a generic parser that can handle multiple data formats or a collection of specialized processors for different file types (PDF parser, spreadsheet parser, image OCR, etc.). The processing service takes an input (reference to a raw file in storage, or a message containing data) and performs the extraction and transformation logic. It uses the rules of the Concept2Cure data model to map raw information into structured records. The output of this service is clean, validated data ready to be saved. This service might also call on AI sub-components – for example, using an NLP model (possibly even an AI like Claude or GPT) to assist in understanding unstructured text fields. The modular design allows plugging in such AI helpers if needed for tasks like entity recognition or summarization (especially given the platform's AI focus). After processing each item, it hands off the structured result to the loading stage and logs the outcome (success or any parsing errors).

- **Data Loading & Storage Module:** This component is responsible for taking the structured data from the processor and inserting it into persistent storage. It could be implemented as a lightweight API service (e.g., a RESTful endpoint that accepts data batch and writes to DB) or simply as a database connector library used by the processing service. In a microservice design, keeping it separate can be beneficial for scaling and security – the loader service can encapsulate all logic about how to interact with the database or data lake. It ensures the data conforms to the schema, handles any upsert or merging logic (for updates to existing entries), and commits the data. Once data is stored, it may emit an event like "data X loaded successfully" which can trigger the next step. The storage itself could be a **polyglot persistence layer**: e.g., a relational database for core structured data, and maybe a document store or blob storage for any semi-structured content, as well as a time-series or graph store if needed [13] [9]. The loader abstracts these details and provides a unified interface to store new data.

- **Intelligence Cortex Integration:** This is a higher-level service or set of routines that update the platform's **"brain"**. In Concept2Cure, this cortex likely powers advanced search and AI features across all ingested data (spanning scientific, clinical, regulatory domains). The integration service takes the newly loaded data and performs tasks to integrate it into this layer. For example, it might: generate or update knowledge graph nodes/edges, index text in a full-text search engine, or calculate embeddings for use in a vector database for semantic search. It could also trigger model retraining or update dashboards. Essentially, this component makes sure the **new data is not just stored, but also connected to existing knowledge** and ready to be used by end-user features (like an AI assistant answering questions using the data). In an AI-native platform, this step is crucial for **contextualizing data** – linking it with similar entities, deduplicating if the same fact appears twice, and applying ontologies or taxonomies. (For instance, if a new FDA guideline is ingested, the cortex might link it to related regulations or tag it under certain categories for easy retrieval by the system's QA assistant.)

- **Coordinator & Orchestration:** While each service can operate independently, we typically include an orchestration mechanism to coordinate the whole pipeline. This could be a simple **scheduler or controller service** that triggers harvesters at certain intervals and ensures the system is healthy. More advanced setups might use an orchestration framework like Apache Airflow or Kubernetes

operators to manage workflows. The coordinator tracks which files have been processed and handles retries or error routing if something fails. Importantly, the orchestration is what enables the **"constant farming"**: e.g., a scheduler wakes up each harvester microservice periodically (or they self-schedule via cron jobs or event subscriptions), ensuring a continuous loop of data collection. The orchestrator can also handle scaling signals – if backlog grows, it might spin up additional processor instances (farmers) to handle the load. All inter-service communication can happen through well-defined APIs or asynchronously via a message broker. For instance, a harvester might POST a "new file available" event to a processing service's API, or drop a message in a queue that the processor service is subscribed to. Using asynchronous messaging (Kafka topics, RabbitMQ queues, etc.) is a common way to keep the pipeline loosely coupled and scalable [10] [12], as noted above.

Each of these components can be developed and deployed independently – for example, as Docker containers within a Kubernetes cluster, or as serverless functions triggered by events. The **microservice architecture** not only makes the system more maintainable but also aligns with the platform's need for an AI-native, cloud-friendly solution. It allows using the best tool for each job (for example, a Python service with NLP libraries for text parsing, a Java service for high-speed database writes, etc.) and then integrating them via APIs. This approach is in line with modern data pipeline best practices, where different stages of data ingestion (ingestion, transformation, storage, indexing) are handled by specialized components working in unison [3] [8].

Continuous Operation and Scalability

To ensure the harvester "works constantly," the design employs a **continuous loop and scalable infrastructure**:

- **Event-Driven Updates:** Instead of waiting for a batch, the system can react to events. For example, if the source systems can emit notifications (webhooks, messages) when new data is available, those can directly trigger the harvester for near-real-time ingestion. If not, the harvesters will poll on a frequent schedule. In either case, new data flows in as soon as possible. The use of message queues helps buffer and distribute work, so components are never overwhelmed by bursts of incoming files [14].

- **Parallel "Farmers":** Multiple harvester services can run in parallel for different sources (or even for the same source divided by segments). Similarly, multiple processing instances can work on different files concurrently. This is akin to having a **farm of workers** fetching and processing data simultaneously, increasing throughput. If the volume of data increases, the system can scale horizontally by adding more worker instances (more containers or processes), as long as the underlying messaging system and databases can handle the load. Because producers and consumers are decoupled, you can scale them independently – e.g., if processing is the bottleneck, add more processing instances without changing the harvesters [10].

- **Robust Error Handling:** The pipeline will inevitably encounter errors (e.g., a source is temporarily unreachable, or a file format is unrecognized). Each microservice should handle errors gracefully and not crash the entire pipeline. For instance, if a harvester fails to fetch from a source, it can log and retry later without blocking others. If the processor hits a parsing error on a particular file, it could quarantine that file for manual review and move on to the next message. The orchestrator or

message queue can implement **retries and dead-letter queues** for messages that repeatedly fail processing, ensuring the pipeline keeps moving and no single bad file stops the harvest.

- **State Management:** For continuous operation, harvesters need to remember what they've already fetched (to avoid duplicates). This can be handled via simple state markers (e.g., last timestamp fetched, last file ID seen) either stored in a small database or as metadata in the platform. Each harvester updates its state after successful fetch. The design should ensure idempotency as well – if a file gets processed twice by accident, the loader can detect duplicate entries (perhaps using unique IDs) and avoid creating duplicate records. This makes the pipeline robust to hiccups or restarts.

- **Monitoring and Observability:** To keep the pipeline healthy 24/7, include monitoring on each component. Metrics like number of files fetched, processing latency, error rates, queue lengths, etc., should be tracked. If a backlog is growing (indicating a slowdown in one stage), alerts can be raised or auto-scaling can be triggered. Logging across services should be aggregated (using a tool like ELK stack or cloud monitoring) so that it's easy to trace the journey of a specific file through the pipeline for debugging. Data observability platforms can also be integrated to watch data quality as it flows [4] [15], which is especially important in a regulatory intelligence context (e.g., ensuring no data is lost or corrupted during transformation).

Implementation Considerations

While the design is technology-agnostic, implementing this in a **GitHub Codespaces** environment with modern AI-assisted tools is feasible and efficient. Developers can use GitHub Codespaces for a consistent dev environment and leverage **Copilot** or similar AI coding assistants to scaffold the microservices. For example, Copilot can help generate boilerplate for API endpoints or database connectors, speeding up development. The mention of a "sub agent Claude opus 4.5" suggests possibly using an AI like **Claude** for certain intelligent processing tasks. Indeed, one can integrate an AI agent within the pipeline – for instance, an NLP AI service that summarizes documents or extracts specific insights could be one of the processing steps. This would align with Concept2Cure's AI-native approach, injecting advanced intelligence into the data scrubbing phase.

For APIs, each microservice can expose RESTful endpoints (or GraphQL, gRPC, etc.) to interact if needed – e.g., an endpoint to manually trigger a harvest, or to query the status of a processed file. However, much of the inter-service communication in the automated pipeline will happen via the message bus and database, not direct API calls, to keep things asynchronous. The **OpenAPI** specification can be used to define these service interfaces for consistency and discoverability [16], which is helpful in a microservice ecosystem. All services would be containerized (Docker), allowing deployment on a container platform. Kubernetes could then ensure they're always running and scale them based on CPU/memory or queue depth triggers.

Security is also a concern: since we are dealing with potentially sensitive regulatory and clinical data, all data transfers should be encrypted, and appropriate authentication/authorization should guard the API endpoints. Each microservice should handle data access according to the principle of least privilege (the harvester can read from sources but not write, the loader can write to the database but maybe not read everything, etc.). Audit logs of what data was pulled and processed when should be maintained for compliance.

Finally, it's worth noting that this design is flexible. It can support **batch processing or streaming** as needed. In some cases, daily batch imports might suffice (the harvester triggers nightly), while in others (like real-time monitoring or fraud detection use-cases) a streaming approach with Kafka would be more appropriate [17] [18]. The architecture can accommodate either by swapping out the scheduling mechanism. The use of farmers (multiple small services) means the pipeline can evolve: new data sources can be added by spinning up new harvester services; new processing logic can be deployed as a new microservice for a specific file type, without rewriting the whole system.

Conclusion

The proposed automated data harvester for Concept2Cure is a **modular, multi-service "farm"** that continuously feeds the platform's data needs. By separating concerns into dedicated microservices – from source-specific harvesters to processing, loading, and cortex integration – the system achieves a high degree of flexibility and resilience. Each stage of the pipeline is optimized for its task, and together they transform raw files into actionable intelligence. This design ensures that incoming data (scientific papers, regulatory documents, clinical data, etc.) is **quickly harvested, cleansed, and integrated** into Concept2Cure's intelligence cortex with minimal manual intervention.

Not only does this fulfill the immediate need of keeping the data model populated, it also sets up the platform for future growth: new "farmers" can be added as the scope of data expands, and the whole pipeline can scale horizontally to handle larger volumes. In essence, we have a **constant data farming system** – much like a fleet of autonomous harvesters on a farm – working in concert to gather knowledge, process it into a nutritious form, and feed it into the brain of the Concept2Cure platform. This will empower end users with up-to-date, comprehensive intelligence at their fingertips, accelerating the path from **concept to cure**.

Sources:

- Monte Carlo Data, *"How to Design a Modern, Robust Data Ingestion Architecture"* – describes typical data ingestion steps (source extraction, transformation, loading, validation, etc.) and tools [3] [4].
- Monte Carlo Data, *"Data Pipeline Architecture Explained"* – importance of orchestration for continuous, reliable data import in pipelines [1].
- SWIM Decoupled Architecture (NSF research) – example of microservices with polyglot persistence and a semantic knowledge base for integrated data intelligence [13] [9].
- EmergentMind, *"Message-Queue-Based Decoupling"* – explains how using message queues enables asynchronous, independent scaling of producers and consumers in a pipeline [10] [12].

---

[1] [3] [4] [5] [6] [7] [8] [15] [17] [18] How To Design A Modern, Robust Data Ingestion Architecture
https://www.montecarlodata.com/blog-design-data-ingestion-architecture/

[2] [9] [13] [16] Microsoft Word - Supporting-Regional-Water-NSF.docx
https://par.nsf.gov/servlets/purl/10408837

[10] [11] [12] [14] Message-Queue-Based Decoupling
https://www.emergentmind.com/topics/message-queue-based-decoupling